Quickstart

Prerequisites

Make sure you've followed the base quickstart instructions for the Agents SDK, and set up a virtual environment. Then, install the optional voice dependencies from the SDK:

```
pip install 'openai-agents[voice]'
```

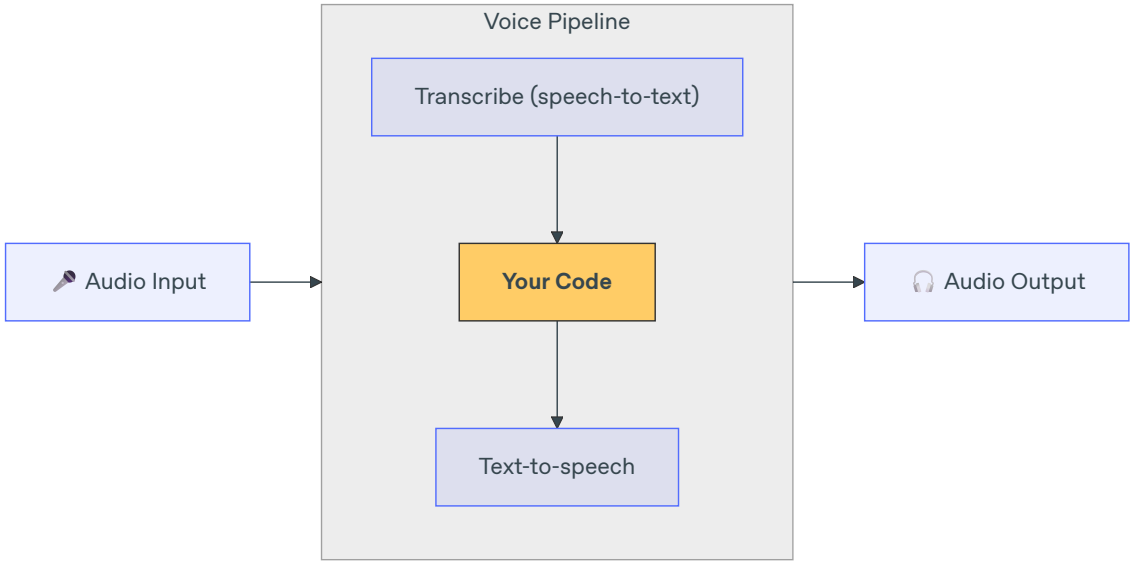
The demo code below also uses `sounddevice` for microphone and speaker I/O, which is not part of the `voice` extra:

```
pip install sounddevice
```

Concepts

The main concept to know about is a `VoicePipeline`, which is a 3 step process:

1. Run a speech-to-text model to turn audio into text.
2. Run your code, which is usually an agentic workflow, to produce a result.
3. Run a text-to-speech model to turn the result text back into audio.



Agents

First, let's set up some Agents. This should feel familiar to you if you've built any agents with this SDK. We'll have two Agents, a configured handoff, and a tool.

```
import random

from agents import Agent
from agents.decorators import tool
from agents.extensions.handoff_prompt import prompt_with_handoff_i


@tool
def get_weather(city: str) -> str:
    """Get the weather for a given city."""
    print(f"[debug] get_weather called with city: {city}")
    choices = ["sunny", "cloudy", "rainy", "snowy"]
    return f"The weather in {city} is {random.choice(choices)}."


spanish_agent = Agent(
    name="Spanish",
    handoff_description="A Spanish-speaking agent.",
    instructions=prompt_with_handoff_instructions(
        "You're speaking to a human, so be polite and concise. Spe
    ),
    model="gpt-5.6-sol",
)
```

```
agent = Agent(
    name="Assistant",
    instructions=prompt_with_handoff_instructions(
        "You're speaking to a human, so be polite and concise. If
    ),
    model="gpt-5.6-sol",
    handoffs=[spanish_agent],
    tools=[get_weather],
)
```

Voice pipeline

We'll set up a simple voice pipeline, using `SingleAgentVoiceWorkflow` as the workflow.

```
from agents.voice import SingleAgentVoiceWorkflow, VoicePipeline
pipeline = VoicePipeline(workflow=SingleAgentVoiceWorkflow(agent))
```

Run the pipeline

```
import numpy as np
import sounddevice as sd
from agents.voice import AudioInput

# For simplicity, we'll just create 3 seconds of silence
# In reality, you'd get microphone data
buffer = np.zeros(24000 * 3, dtype=np.int16)
audio_input = AudioInput(buffer=buffer)

result = await pipeline.run(audio_input)

# Create an audio player using `sounddevice`
player = sd.OutputStream(samplerate=24000, channels=1, dtype=np.in
player.start()

# Play the audio stream as it comes in
async for event in result.stream():
    if event.type == "voice_stream_event_audio":
        player.write(event.data)
```

Put it all together

```python
import asyncio
import random

import numpy as np
import sounddevice as sd

from agents import Agent
from agents.decorators import tool
from agents.voice import (
    AudioInput,
    SingleAgentVoiceWorkflow,
    VoicePipeline,
)
from agents.extensions.handoff_prompt import prompt_with_handoff_i


@tool
def get_weather(city: str) -> str:
    """Get the weather for a given city."""
    print(f"[debug] get_weather called with city: {city}")
    choices = ["sunny", "cloudy", "rainy", "snowy"]
    return f"The weather in {city} is {random.choice(choices)}."


spanish_agent = Agent(
    name="Spanish",
    handoff_description="A Spanish-speaking agent.",
    instructions=prompt_with_handoff_instructions(
        "You're speaking to a human, so be polite and concise. Spe
    ),
    model="gpt-5.6-sol",
)

agent = Agent(
    name="Assistant",
    instructions=prompt_with_handoff_instructions(
        "You're speaking to a human, so be polite and concise. If
    ),
    model="gpt-5.6-sol",
    handoffs=[spanish_agent],
    tools=[get_weather],
)
```

```python
async def main():
    pipeline = VoicePipeline(workflow=SingleAgentVoiceWorkflow(age
    buffer = np.zeros(24000 * 3, dtype=np.int16)
    audio_input = AudioInput(buffer=buffer)

    result = await pipeline.run(audio_input)

    # Create an audio player using `sounddevice`
    player = sd.OutputStream(samplerate=24000, channels=1, dtype=n
    player.start()

    # Play the audio stream as it comes in
    async for event in result.stream():
        if event.type == "voice_stream_event_audio":
            player.write(event.data)


if __name__ == "__main__":
    asyncio.run(main())
```

If you run this example, the agent will produce spoken audio for you to hear! Check out the example in examples/voice/static to see a demo where you can speak to the agent yourself.